

Abschlussprüfung Sommer 2026

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur betrieblichen Projektarbeit

Thema Entwicklung des AntispamAdminCenter

Mandantenfähige Webanwendung zur Verwaltung von Mail- und IP-Freigaben

Abgabetermin: Frankfurt, den 29.04.2026

Prüfungsbewerber:

Klaus Bindemann
Kastanienweg 2b
15377 Oberbarnim



villadata
systemhaus GmbH

Ausbildungsbetrieb:

AVADO - VILLADATA Systemhaus GmbH
Am Flugplatz 12a
15344 Strausberg

Inhaltsverzeichnis

Abkürzungsverzeichnis	III
1 Das Thema und Problemstellung	1
1.1 Zielstellung der Projektarbeit	1
1.2 Ist-Analyse	2
1.3 Soll-Konzept	2
1.4 Projektabgrenzung	2
1.5 Projektorganisation und Zeitplanung	3
2 Technisch-wirtschaftlicher Variantenvergleich	3
2.1 Technische Bewertung der Varianten	3
2.2 Wirtschaftliche Bewertung	4
2.3 Nutzwertanalyse und Entscheidung	5
3 Durchführung und Ergebnisse	5
3.1 Architektur und Zielplattform	5
3.2 Entwurf von Datenmodell und Rechtestruktur	6
3.3 Entwurf der Benutzeroberfläche	6
3.4 Sicherheitskonzept und Auditierung	6
3.5 Schnittstellen und Datenfluss	6
3.6 Implementierung der wichtigsten Funktionen	7
3.7 Implementierung der Benutzer- und Mandantenverwaltung	7
3.8 Erzielte Projektergebnisse	7
4 Qualitätssicherung	8
4.1 Soll-Ist-Vergleich von Leistung, Zeit und Inhalt	8
4.2 Bewertung der Testergebnisse	9
5 Kundendokumentation	9
6 Fazit	9
Literatur- und Quellenverzeichnis	11
Persönliche Erklärung	12
A Anhang	i
A.1 Zeitnachweis	i
A.2 Wirtschaftlichkeit	ii
A.3 Variantenvergleich	iii
A.4 Lastenheft	iv
A.5 Pflichtenheft	vi
A.6 Use Case-Diagramm	viii

A.7 Datenbankmodell	ix
A.8 Klassendiagramm	x
A.9 Komponentendiagramm	xi
A.10 Sequenzdiagramm Login	xii
A.11 Kundenhinweis	xiii
A.12 Quelltextauszug zu Datenschutz und Sicherheit	xiv

Abkürzungsverzeichnis

AD Active Directory

API Application Programming Interface

CRUD Create, Read, Update, Delete

GUI Graphical User Interface

HTML Hypertext Markup Language

HTTP Hypertext Transfer Protocol

IP Internet Protocol

ORM Object-Relational Mapping

SQL Structured Query Language

UI User Interface

UML Unified Modeling Language

1 Das Thema und Problemstellung

Der Auftraggeber ist die AVADO - VILLADATA Systemhaus GmbH, ein Systemhaus für IT-Dienstleistungen und den Betrieb von Kundeninfrastrukturen. Zu den Kunden gehören Unternehmen, deren Mail-Systeme entweder auf einer lokalen Active-Directory-Umgebung oder auf einer Cloud-basierten Verzeichnisstruktur beruhen. Für diese Kunden werden unter anderem Mail- und IP-Freigaben betreut, damit nachgelagerte Antispam-Systeme eingehende Nachrichten korrekt zuordnen und bewerten können.

Bei Kunden mit lokaler Active-Directory-Umgebung lief dieser Prozess bereits weitgehend automatisiert. Ein Cronjob rief regelmäßig per Skript die benötigten Mail-Adressen und IP-Adressen aus dem Kundensystem ab und legte sie in einer für die Antispam-Infrastruktur geeigneten Datei ab. Bei Cloud-basierten Kunden standen diese Daten dagegen nicht in gleicher Form automatisiert zur Verfügung. Die Freigaben wurden deshalb manuell auf Shell-Ebene unter Linux gepflegt. Informationen waren auf mehrere Datenquellen verteilt und Rückfragen mussten häufig im direkten Austausch geklärt werden.

Problematisch war vor allem, dass nicht jeder Administrator sicher auf Shell-Ebene arbeiten kann oder die dafür nötigen Linux-Kenntnisse im gleichen Umfang mitbringt. Zusätzlich fehlte eine einheitliche Oberfläche, über die Bearbeitung, Zuständigkeiten und Änderungen klar abgebildet wurden.

1.1 Zielstellung der Projektarbeit

Ziel des Projekts war die Entwicklung eines *AntispamAdminCenter* mit Weboberfläche. Es handelt sich um eine webbasierte Verwaltungsanwendung für die zentrale Pflege von Firmen, Benutzern, Mail-Adressen und IP-Adressen. Die Anwendung sollte von internen Administratoren und autorisierten Firmenbenutzern genutzt werden können. Dabei musste klar getrennt bleiben, welche Daten zu welcher Firma gehören und wer sie bearbeiten darf. Vertraulichkeit, Authentizität, Integrität und Verfügbarkeit waren dabei wichtige Anforderungen.

Aus dem Projektauftrag ergaben sich folgende Kernziele:

- zentrale und einheitliche Pflege aller fachlich relevanten Stammdaten
- rollenbasiertes Berechtigungskonzept mit Mandantentrennung
- abgesicherte Anmeldung mit Session-Verwaltung und Passwort-Hashing
- nachvollziehbare Protokollierung fachlicher Änderungen
- Bereitstellung verlässlicher Daten für nachgelagerte Mail-Systeme

Das Projektergebnis sollte nicht nur die Bearbeitungszeit senken, sondern vor allem die Qualität der Datenhaltung verbessern. Die Pflege sollte einfacher, einheitlicher und für andere Administratoren besser nachvollziehbar werden.

1.2 Ist-Analyse

Im Ist-Zustand wurden die Mail- und IP-Freigaben für Cloud-basierte Kunden ohne zentrale Fachanwendung verwaltet. Die benötigten Informationen lagen verteilt vor, zum Beispiel in bestehenden Kundendaten, technischen Konfigurationsdateien und internen Abstimmungen. Änderungen wurden durch Administratoren auf Shell-Ebene vorgenommen. Eine einheitliche Oberfläche für das Anlegen, Bearbeiten und Prüfen der Freigaben gab es nicht.

Auch die Zuordnung zu Firmen und Benutzern war nur indirekt aus den vorhandenen Informationen ableitbar. Dadurch musste bei Pflegevorgängen häufig manuell geprüft werden, zu welchem Kunden ein Eintrag gehört, wer die Änderung angefordert hat und ob die jeweilige Person zur Änderung berechtigt ist. Die fachlichen Daten selbst waren überschaubar, der Prozess hing jedoch stark von Erfahrung, Linux-Kenntnissen und dem Wissen einzelner Administratoren ab.

Aus der Analyse ergaben sich mehrere Schwachstellen. Es fehlte ein zentraler Ort für freigegebene Mail- und IP-Einträge, eine klare Mandantentrennung, eine einheitliche Eingabeprüfung und eine zuverlässige Änderungshistorie. Dadurch entstanden wiederkehrende Rückfragen, manuelle Kontrollaufwände und vermeidbare Fehlerquellen. Außerdem war nur eingeschränkt nachvollziehbar, wer eine Freigabe angelegt, geändert oder entfernt hatte.

1.3 Soll-Konzept

Im Soll-Zustand soll eine zentrale Webanwendung den gesamten Pflegeprozess unterstützen. Firmen, Benutzer, Mail-Adressen und IP-Adressen sollen an einer Stelle verwaltet und über einheitliche Formulare gepflegt werden. Rollen und Firmenzuordnung steuern, welche Daten angezeigt und geändert werden dürfen. Fachliche Änderungen werden im Audit-Log festgehalten, damit sie später nachvollzogen werden können.

Für die Umsetzung wurde eine schlanke Webanwendung auf Basis von FastAPI, SQLAlchemy und SQLite vorgesehen. Diese Auswahl passte zum Projektumfang, weil die Anwendung damit überschaubar blieb und sich die benötigten Funktionen gut testen ließen.

1.4 Projektabgrenzung

Im Projekt wurde die Verwaltungsanwendung selbst umgesetzt. Dazu gehören die Anmeldung, das Rollenmodell, die Pflege von Firmen, Benutzern, Mail-Adressen und IP-Adressen sowie das Audit-Log für Änderungen.

Nicht Teil des Projekts ist der spätere Produktivbetrieb auf mehreren Servern. Auch die eigentliche Spam-Filterung und eine vollständige Übernahme alter Daten aus anderen Systemen wurden nicht umgesetzt. Die angebundenen Mail-Systeme bleiben unverändert und sind nicht Teil dieser Projektarbeit.

Diese Abgrenzung war notwendig, damit der Umfang in den vorgegebenen 80 Stunden umsetzbar bleibt.

1.5 Projektorganisation und Zeitplanung

Die Projektdurchführung erfolgte im für die FIAE-Projektarbeit vorgesehenen Zeitbudget von 80 Stunden. Zur Steuerung des Vorhabens wurde die Arbeit in Analyse, Soll-Konzept, Entwurf, Implementierung, Test, Inbetriebnahmevorbereitung und Dokumentation gegliedert. So war von Anfang an klar, wie viel Zeit für Analyse, Entwicklung, Tests und Dokumentation zur Verfügung steht.

Projektphase	Stunden
Analyse der Ausgangssituation und Anforderungsklä- rung	8
Soll-Konzept und Variantenvergleich	10
Entwurf von Datenmodell, Rollenmodell und Oberflä- che	12
Implementierung der Anwendung	28
Tests, Fehlerkorrekturen und Abnahmevorbereitung	10
Kundendokumentation und Projektdokumentation	12
Summe	80

Tabelle 1: Geplante Zeitaufteilung für das Projekt

Die Umsetzung erfolgte schrittweise. Nach der Erhebung der Anforderungen wurden die Teilbereiche Firmen, Benutzer, Mail-Adressen, IP-Adressen, Authentifizierung und Audiotisierung getrennt betrachtet und danach umgesetzt. Rückmeldungen aus dem internen IT-Umfeld konnten dadurch direkt in den Entwurf und die Implementierung einfließen.

Materiell wurde ein vorhandener Entwicklungsarbeitsplatz mit Python 3.12, FastAPI, SQL-Model, SQLite und einer lokalen Testumgebung genutzt. Externe Lizenzkosten fielen für die im Projekt eingesetzten Kernkomponenten nicht an.

2 Technisch-wirtschaftlicher Variantenvergleich

Für das Projekt wurden drei Lösungsvarianten betrachtet. Die erste Variante war die Beibehaltung der manuellen Pflege auf Shell-Ebene. Die zweite Variante war die Einführung einer allgemeinen Standardlösung für Stammdaten- und Freigabeverwaltung. Die dritte Variante war die Entwicklung einer betriebsspezifischen Webanwendung. Die Entscheidung wurde sowohl fachlich als auch wirtschaftlich bewertet.

2.1 Technische Bewertung der Varianten

Die manuelle Pflege verursacht zunächst keine Einführungskosten, löst aber keines der beschriebenen Probleme. Der Prozess bleibt weiterhin von einzelnen Administratoren ab-

hängig. Außerdem fehlen eine saubere Trennung der Firmen, eine nachvollziehbare Änderungshistorie und eine einheitliche Prüfung der Eingaben.

Eine Standardlösung kann manuellen Aufwand grundsätzlich reduzieren, passt aber nicht automatisch zum vorhandenen Ablauf. Im Projekt geht es nicht um eine beliebige Stammdatenpflege, sondern um Firmen, Mail- und IP-Freigaben, Rollen, Anmeldung und Nachvollziehbarkeit. Bei einer Fremdlösung wären deshalb Anpassungen und Abstimmungen zu erwarten gewesen. Außerdem hätte das Rollenmodell wahrscheinlich an die Standardlösung angepasst werden müssen und nicht umgekehrt.

Die Eigenentwicklung deckt genau die benötigten Funktionen ab und kann am vorhandenen Ablauf ausgerichtet werden. Die Anwendung bleibt überschaubar, weil nur die Funktionen umgesetzt werden, die für die Pflege der Cloud-basierten Kunden wirklich benötigt werden. Der zusätzliche Entwicklungsaufwand passt in den geplanten Zeitrahmen und vermeidet eine unnötige Abhängigkeit von einem Fremdprodukt.

Kriterium	Manuell	Standardlösung	Eigenentwicklung
Mandantentrennung	schwach	mittel	hoch
Fachprozessabbildung	schwach	mittel	hoch
Nachvollziehbarkeit von Änderungen	schwach	mittel	hoch
Anpassbarkeit	niedrig	mittel	hoch
Betriebliche Einführbarkeit	hoch	mittel	hoch

Tabelle 2: Technischer Vergleich der Lösungsvarianten

2.2 Wirtschaftliche Bewertung

Die wirtschaftliche Betrachtung orientiert sich an typischen Pflegevorgängen im Administrationsalltag. Für die Kalkulation wurden 60 Änderungsvorgänge pro Monat angesetzt. Vor der Projektdurchführung dauerte ein Vorgang im Mittel rund 10 Minuten, weil Informationen zusammengesucht, Zuständigkeiten geklärt und Eingaben manuell nachgehalten werden mussten. Mit der neuen Anwendung sinkt der Aufwand auf rund 4 Minuten, weil die Daten an einer Stelle gepflegt werden und weniger Rückfragen entstehen.

Kennzahl	Wert
Pflegevorgänge pro Monat	60
Bearbeitungszeit vorher je Vorgang	10 min
Bearbeitungszeit nachher je Vorgang	4 min
Zeitersparnis je Vorgang	6 min
Zeitersparnis pro Monat	360 min = 6 h

Tabelle 3: Grundlage der Wirtschaftlichkeitsbetrachtung

Für die Projektkosten wurde kein Stundenlohn, sondern ein interner kalkulatorischer Verrechnungssatz angesetzt. Für die 80 Projektstunden wurden 65 € pro Stunde angesetzt. Zusätzlich wurden sechs Stunden fachliche Begleitung und Abnahme mit 85 € pro Stun-

de bewertet. Daraus ergeben sich Projektkosten von 5710 €. Eine Detailrechnung ist im Anhang A.2: Wirtschaftlichkeit dargestellt.

Bei einer monatlichen Einsparung von 6 Stunden und einem kalkulatorischen internen Satz von 65 € ergibt sich ein wirtschaftlicher Nutzen von 390 € pro Monat. Die Projektkosten amortisieren sich damit nach rund 14,6 Monaten. Die Amortisationsdauer ist für eine intern genutzte Verwaltungsanwendung vertretbar. Zusätzlich zur Zeitersparnis werden Fehlerquellen reduziert, Änderungen besser nachvollziehbar und die Firmen klarer getrennt.

2.3 Nutzwertanalyse und Entscheidung

Da die Bearbeitungszeit allein nicht für die Entscheidung ausreicht, wurde zusätzlich eine Nutzwertanalyse durchgeführt. Bewertet wurden die Kriterien Nachvollziehbarkeit, Sicherheit, fachliche Eignung, Bedienbarkeit und Wartbarkeit. Die gewichtete Auswertung ist im Anhang A.3: Variantenvergleich enthalten.

Die manuelle Pflege erzielt den niedrigsten Nutzwert, weil sie den bestehenden Zustand im Wesentlichen beibehält. Die Standardlösung verbessert zwar die Bedienung, müsste aber erst an den konkreten Prozess angepasst werden. Die Eigenentwicklung erzielt den höchsten Nutzwert, weil sie zum vorhandenen Ablauf passt und ohne unnötige Zusatzfunktionen eingeführt werden kann. Aus diesen Gründen wurde die Eigenentwicklung als wirtschaftlich und fachlich sinnvollste Variante ausgewählt.

Für die Entscheidung war wichtig, dass nicht nur die reine Bearbeitungszeit betrachtet wurde. Fehlerhafte Freigaben oder unklare Zuständigkeiten können im Mail-Umfeld direkte Auswirkungen auf den Betrieb haben. Deshalb wurde der höhere Anfangsaufwand der Eigenentwicklung akzeptiert.

3 Durchführung und Ergebnisse

3.1 Architektur und Zielplattform

Die Anwendung wurde als Webanwendung auf Basis von Python 3.12, FastAPI und SQL-Model geplant. Der Zugriff erfolgt über einen Browser, sodass auf den Arbeitsplätzen keine eigene Client-Installation notwendig ist. Das passt zum internen Einsatz, weil Administratoren die Anwendung zentral aufrufen können und nicht mehr direkt auf der Shell arbeiten müssen.

Die Anwendung ist in vier Bereiche gegliedert: Datenmodell, Fachlogik, Web-API und Templates. Im Quellcode entsprechen diese Bereiche den Ordnern `app/models`, `app/services`, `app/web/api` und `app/web/templates`. Dadurch ist klar, wo Daten gespeichert, Regeln geprüft und Inhalte angezeigt werden. Diese Trennung erleichtert die Wartung und die Tests, weil die Regeln nicht in einzelnen Masken verteilt sind. Ein Komponentendiagramm ist im Anhang A.9: Komponentendiagramm dargestellt.

3.2 Entwurf von Datenmodell und Rechtestruktur

Das Datenmodell umfasst Firmen, Benutzer, Mail-Adresse und IP-Adresse. Ergänzt werden diese durch Tabellen für Audit- und Login-Daten. Damit lässt sich später nachvollziehen, welche fachlichen Änderungen durchgeführt wurden und welche Anmeldevorgänge sicherheitsrelevant waren.

Im Berechtigungskonzept wurden globale und firmenbezogene Rechte getrennt. Root- und Admin-Rollen dürfen administrative Aufgaben für mehrere Firmen ausführen. Einfache Benutzer bleiben dagegen auf die Firmen beschränkt, denen sie zugeordnet sind. Diese Trennung ist wichtig, damit Kundendaten nicht versehentlich firmenübergreifend sichtbar oder bearbeitbar werden. Das Datenbankmodell ist im Anhang A.7: Datenbankmodell dokumentiert.

3.3 Entwurf der Benutzeroberfläche

Die Benutzeroberfläche wurde bewusst einfach gehalten. Im Mittelpunkt stand, dass Daten schnell und mit möglichst wenigen Fehlern erfasst werden können. Die Navigation führt direkt zu den Kernbereichen Dashboard, Firmen, Benutzer, Mail-Adressen, IP-Adressen und Audit-Log. Die einzelnen Masken sind formularbasiert aufgebaut und orientieren sich an den täglichen Pflegevorgängen des Fachprozesses.

Fehler werden direkt an den betroffenen Eingabefeldern angezeigt. Dadurch fallen falsche E-Mail-Adressen, ungültige IP-Adressen oder unvollständige Stammdaten sofort auf und müssen nicht später manuell gesucht werden. Die einheitliche Bedienung soll den Einstieg erleichtern und fehlerhafte Eingaben reduzieren.

3.4 Sicherheitskonzept und Auditierung

Neben der fachlichen Bedienung wurde ein Sicherheitskonzept entworfen. Dazu gehören Session-Login, Passwort-Hashing, Login-Audit, Begrenzung wiederholter Fehlanmeldungen sowie kontobezogene Sperrmechanismen. Diese Funktionen sollen verhindern, dass Verwaltungsfunktionen ohne Berechtigung genutzt werden.

Das Audit-Log wurde als eigener Teil der Anwendung vorgesehen. Jede relevante Änderung an Firmen, Benutzern, Mail-Adressen und IP-Adressen wird mit Zusatzinformationen protokolliert. Dadurch können fehlerhafte Pflegevorgänge nachvollzogen und bewertet werden. Für ausgewählte Szenarien ist außerdem ein Rollback vorgesehen, damit unbeabsichtigte Änderungen kontrolliert zurückgenommen werden können.

3.5 Schnittstellen und Datenfluss

Die Anwendung stellt sowohl HTML-basierte Oberflächen als auch API-Endpunkte bereit. Dadurch können Daten direkt über den Browser gepflegt werden. Die verwalteten

Mail- und IP-Informationen stehen gleichzeitig strukturiert für nachgelagerte Verarbeitungsschritte bereit. So müssen die Daten nicht doppelt gepflegt werden und können einfacher weiterverarbeitet werden.

Im Datenfluss beginnt jeder Pflegevorgang mit der Authentifizierung des Benutzers. Danach prüft die Anwendung Rolle und Firmenzuordnung, validiert die Eingaben, schreibt die Änderung in die Datenbank und erzeugt bei fachlich relevanten Aktionen einen Audit-Eintrag. So wird bei jedem Vorgang geprüft, ob der Benutzer die Änderung durchführen darf und ob sie später nachvollzogen werden kann.

3.6 Implementierung der wichtigsten Funktionen

Die Umsetzung erfolgte schrittweise entlang der fachlichen Teilbereiche. Zunächst wurden die Datenklassen für Firmen, Benutzer, Mail-Adressen, IP-Adressen und Audit-Daten angelegt. Danach folgten die Funktionen für Validierung, Benutzerverwaltung, Rollenprüfung und die Pflegeprozesse. So konnte jeder Teilbereich einzeln umgesetzt und getestet werden, bevor er in der Weboberfläche genutzt wurde.

Ein Schwerpunkt lag auf der Anmeldung und Absicherung der Benutzerkonten. Die Anwendung nutzt Session-Login, Passwort-Hashing mit Argon2, Login-Audit sowie Schutzmechanismen gegen wiederholte Fehlanmeldungen. Damit wird verhindert, dass sensible Verwaltungsfunktionen ohne ausreichenden Schutz erreichbar sind.

3.7 Implementierung der Benutzer- und Mandantenverwaltung

Die Benutzer- und Mandantenverwaltung ist ein zentraler Teil der Anwendung. Firmen können angelegt, geändert und getrennt verwaltet werden. Benutzer erhalten Rollen und werden bestimmten Firmen zugeordnet. Auf dieser Grundlage steuert die Anwendung, welche Datensätze jemand sehen oder bearbeiten darf.

Für Mail- und IP-Einträge wurden eigene Verwaltungsfunktionen umgesetzt. Sie prüfen Eingaben und protokollieren Änderungen. Dadurch werden ungültige Daten früh erkannt und die nachgelagerten Mail-Systeme können auf eine einheitliche Datenbasis zugreifen.

3.8 Erzielte Projektergebnisse

Mit Abschluss der Implementierung lag eine lauffähige Webanwendung vor. Firmen, Benutzer, Mail-Adressen und IP-Adressen können zentral verwaltet werden. Fachliche Änderungen werden im Audit-Log dokumentiert und können für definierte Fälle über einen Rollback wiederhergestellt werden.

Zusammenfassend wurde damit nicht nur eine neue Eingabemaske geschaffen. Der bisher manuelle Pflegeprozess wurde in eine strukturierte Anwendung mit Rechtekonzept, Prüfung der Eingaben und Änderungshistorie überführt. Die wichtigsten Quelltextauszü-

ge zu Datenschutz und Sicherheit befinden sich im Anhang A.12: Quelltextauszug zu Datenschutz und Sicherheit.

Während der Implementierung zeigte sich, dass die Trennung zwischen firmenbezogenen und globalen Rechten nicht nur in der Oberfläche berücksichtigt werden darf. Die Prüfung muss auch in den fachlichen Funktionen stattfinden, damit geschützte Aktionen nicht über andere Wege ausgeführt werden können. Deshalb wurde die Rechteprüfung in den Service-Funktionen umgesetzt.

4 Qualitätssicherung

Die Qualitätssicherung erfolgte durch automatisierte und manuelle Tests. Mit Pytest wurden vor allem Validierungen, Benutzerverwaltung, Audit-Funktionen und Formularlogik geprüft. Zusätzlich wurden die wichtigsten Arbeitsabläufe direkt über die Weboberfläche getestet, darunter Anmeldung, Rollenvergabe, Mandantentrennung sowie die Pflege von Mail- und IP-Adressen.

Diese Mischung war für das Projekt passend, weil sie sowohl die fachlichen Regeln als auch die Bedienung abdeckt. Die automatisierten Tests helfen dabei, Fehler bei späteren Änderungen schneller zu erkennen. Die manuellen Tests waren zusätzlich notwendig, um Sichtbarkeiten, Navigation und Fehlermeldungen im Zusammenspiel der einzelnen Bereiche zu prüfen. Auszüge aus den sicherheitsrelevanten Kernfunktionen befinden sich im Anhang A.12: Quelltextauszug zu Datenschutz und Sicherheit.

4.1 Soll-Ist-Vergleich von Leistung, Zeit und Inhalt

Die geplanten Funktionen wurden erreicht. Die Anwendung ist mehrmandantenfähig, stellt ein rollenbasiertes Berechtigungskonzept bereit und ermöglicht die zentrale Pflege von Firmen, Benutzern, Mail-Adressen und IP-Adressen. Zusätzlich wurden Audit-Log, Login-Schutz und Rollback für ausgewählte Änderungen umgesetzt.

Aspekt	Soll	Ist
Mandantenfähige Verwaltung	umgesetzt	umgesetzt
Rollen- und Rechtesystem	umgesetzt	umgesetzt
Auditierbare Änderungen	umgesetzt	umgesetzt
Rollback einzelner Änderungen	optional	umgesetzt
Projektzeit	80 h	eingehalten

Tabelle 4: Soll-Ist-Vergleich

Inhaltlich gab es keine grundlegenden Abweichungen vom Projektziel. Im Projektverlauf wurde etwas mehr Wert auf Sicherheit und Nachvollziehbarkeit gelegt. Das war sinnvoll, weil bei einer zentralen Verwaltungsanwendung klar sein muss, wer welche Daten bearbeiten darf und welche Änderungen später nachvollzogen werden können.

4.2 Bewertung der Testergebnisse

Die durchgeführten Tests bestätigten, dass die wesentlichen Pflegeabläufe stabil funktionieren. Besonders wichtig war dabei die Kombination aus Login, Rollenprüfung, Mandantentrennung und Auditierung. Offene Punkte betreffen keine fehlenden Hauptfunktionen, sondern mögliche spätere Erweiterungen wie weitergehende Integrations- oder Lasttests für einen produktionsnahen Ausbau.

5 Kundendokumentation

Die Anwendung wird über einen Webbrowser aufgerufen. Nach erfolgreicher Anmeldung stehen die Menüpunkte Dashboard, Firmen, Benutzer, Mail-Adressen, IP-Adressen und Audit-Log zur Verfügung. Welche Bereiche ein Benutzer sehen oder bearbeiten darf, richtet sich nach seiner Rolle und der zugeordneten Firma.

Neue Firmen, Benutzer sowie Mail- und IP-Einträge werden über die jeweiligen Listenansichten angelegt. Änderungen erfolgen über Bearbeitungsformulare, die Eingaben bereits beim Speichern prüfen. Fehlerhafte Eingaben werden direkt am betroffenen Feld angezeigt. Dadurch kann die Pflege ohne Shell-Zugriff und ohne tiefergehende Linux-Kenntnisse nachvollziehbar erfolgen.

Das Audit-Log dient der Rückverfolgung fachlicher Änderungen. Es zeigt, wann ein Datensatz erstellt, geändert oder gelöscht wurde. Soweit fachlich freigegeben, können Änderungen aus dem Audit-Log heraus zurückgesetzt werden. Ein kurzer Benutzerhinweis mit den wichtigsten Bedienregeln befindet sich zusätzlich im Anhang A.11: Kundenhinweis.

Für die Einführung beim Kunden ist keine umfangreiche Schulung erforderlich. Die Anwendung orientiert sich an den bekannten Objekten des Fachprozesses und verwendet eine einheitliche Bedienung für Listen-, Detail- und Bearbeitungsansichten. Dadurch kann sich die Kundendokumentation auf die Schritte konzentrieren, die im Betrieb wirklich benötigt werden.

Im laufenden Betrieb ist darauf zu achten, dass Berechtigungen nur nach dem tatsächlichen Aufgabenbereich vergeben werden.

6 Fazit

Mit dem *AntispamAdminCenter* wurde eine betriebsnahe Verwaltungsanwendung entwickelt, die den zuvor manuellen Pflegeprozess für Cloud-basierte Kunden zentralisiert. Mail- und IP-Freigaben müssen dadurch nicht mehr auf Shell-Ebene gepflegt werden. Der Nutzen liegt nicht nur in der Zeitersparnis. Die Daten werden einheitlich gepflegt, Rechte sind klarer vergeben und Änderungen lassen sich später nachvollziehen.

Das Projekt hat gezeigt, dass auch ein kleinerer Anwendungsfall sorgfältig umgesetzt werden muss, wenn mehrere Firmen, Rollen und sicherheitsrelevante Vorgaben beteiligt sind. Wenn die Anwendung später produktiv genutzt wird, sollte sie an eine eigene Datenbank angebunden werden. Als nächster Schritt wäre außerdem zu prüfen, ob die Anmeldung an die bestehende Benutzerverwaltung angebunden werden kann zB. Azure AD .

Aus Sicht des Betriebs ist das Ergebnis eine gute Grundlage, um den Prozess weiter zu vereinheitlichen. Die wichtigsten Probleme aus der alten Arbeitsweise wurden reduziert: fehlende Transparenz, uneinheitliche Pflege und die Abhängigkeit von Shell-Kenntnissen.

Literatur- und Quellenverzeichnis

LaTeX-Vorlage Stefan Macke: LaTeX-Vorlage zur IHK-Projektdokumentation für Fachinformatiker Anwendungsentwicklung, lizenziert unter Creative Commons Namensnennung - Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0).

<http://fiae.link/LaTeXVorlageFIAE>

<http://creativecommons.org/licenses/by-sa/4.0/>

Python Python Software Foundation: Python 3.12 Documentation.

<https://docs.python.org/3.12/>

FastAPI FastAPI: Offizielle Dokumentation zum Webframework, Routing, Dependency Injection und API-Aufbau.

<https://fastapi.tiangolo.com/>

SQLModel SQLModel: Offizielle Dokumentation zur Modellierung und Nutzung relationaler Datenbanken in Python.

<https://sqlmodel.tiangolo.com/>

PlantUML PlantUML: Dokumentation zur Erstellung von UML-Diagrammen.

<https://plantuml.com/de/>

Argon2 argon2-cffi: Dokumentation zu Passwort-Hashing mit PasswordHasher.

<https://argon2-cffi.readthedocs.io/en/stable/howto.html>

W3Schools W3Schools: HTML Forms, Grundlagen zu Formularen und Eingabefeldern.

https://www.w3schools.com/html/html_forms.asp

Stack Overflow Stack Overflow: Diskussion zu FastAPI OAuth2 Scope und rollenbasierter Zugriffskontrolle.

<https://stackoverflow.com/questions/64812279/fastapi-oauth2-scope>

Reddit Reddit, r/FastAPI: Diskussion zu FastAPI, SQLModel und API-Aufbau im praktischen Einsatz.

<https://www.reddit.com/r/FastAPI/comments/pgdd6p>

KI-Unterstützung OpenAI: ChatGPT, genutzt zur Informationsbeschaffung, sprachlichen Überarbeitung, Formulierungshilfe und CSS-Angaben.

<https://openai.com/chatgpt/overview/>

Persönliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Projektarbeit selbständig und nur unter Zuhilfenahme der aufgeführten Quellen angefertigt habe. Der vorgesehene zeitliche Rahmen wurde eingehalten.

Unterschrift des Prüfungsteilnehmers

A Anhang

A.1 Zeitnachweis

Tätigkeit	Stunden
Ist-Analyse und Anforderungsklä rung	8
Soll-Konzept und Variantenvergleich	10
Entwurf von Datenmodell und Rollenmodell	8
Entwurf der Weboberfläche und Navigationsstruktur	4
Implementierung Stammdatenverwaltung	10
Implementierung Benutzer- und Rechteverwaltung	8
Implementierung Login-Schutz und Audit-Log	10
Tests, Fehlerkorrekturen und Abnahmevorbereitung	10
Kundendokumentation und Projektdokumentation	12
Summe	80

Tabelle 5: Zeitnachweis

A.2 Wirtschaftlichkeit

Position	Stunden	Satz	Kosten
Projektdurchführung	80	65 €	5200 €
Fachliche Begleitung und Abnahme	6	85 €	510 €
Softwarelizenzen	-	-	0 €
Gesamt			5710 €

Tabelle 6: Detailrechnung der Projektkosten

A.3 Variantenvergleich

Kriterium	Gewicht	Standardlösung	Eigenentwicklung
Passgenauigkeit	5	3	5
Mandantentrennung und Rechteabbildung	5	3	5
Nachvollziehbarkeit und Auditierung	4	3	5
Einführungsaufwand	3	3	4
Wartbarkeit im Betrieb	3	3	4
Gewichteter Nutzwert		51	80

Tabelle 7: Gewichteter Variantenvergleich

A.4 Lastenheft

Das Lastenheft beschreibt die fachlichen Anforderungen des Auftraggebers an das *AntispamAdminCenter*. Ziel ist die zentrale Verwaltung von Firmen, Benutzern, Mail-Adressen und IP-Adressen für die Antispam-nahe Mail-Infrastruktur.

Ausgangssituation

Vor Projektbeginn wurden Freigaben und Zuständigkeiten nicht in einer zentralen Anwendung gepflegt. Relevante Informationen waren auf mehrere Datenquellen verteilt. Dadurch entstanden Mehrfacheingaben, Medienbrüche, Abstimmungsaufwand und eine nur eingeschränkt nachvollziehbare Historie von Änderungen.

Zielsetzung

Die neue Anwendung soll die Pflege vereinheitlichen, die Mandantentrennung sicher abbilden und fachliche Änderungen revisionsfähig protokollieren. Die Lösung soll intern über den Browser nutzbar sein und eine zuverlässige Datenbasis für nachgelagerte Mail-Systeme bereitstellen.

Funktionale Anforderungen

1. Die Anwendung muss Firmen zentral anlegen, ändern, anzeigen und verwalten können.
2. Die Anwendung muss Benutzer mit Rollen und Firmenzuordnung verwalten können.
3. Die Anwendung muss Mail-Adressen firmenbezogen erfassen, validieren und bearbeiten können.
4. Die Anwendung muss IP-Adressen firmenbezogen erfassen, validieren und bearbeiten können.
5. Die Anwendung muss zwischen globalen und mandantenbezogenen Rechten unterscheiden.
6. Die Anwendung muss eine abgesicherte Anmeldung für berechtigte Benutzer bereitstellen.
7. Die Anwendung muss fachliche Änderungen an den verwalteten Objekten in einem Audit-Log protokollieren.
8. Die Anwendung muss die verwalteten Mail- und IP-Daten für nachgelagerte technische Verarbeitung strukturiert bereitstellen.
9. Die Anwendung soll eine einheitliche und leicht verständliche Bedienoberfläche für die tägliche Pflege bieten.

Nichtfunktionale Anforderungen

1. Die Anwendung muss ohne lokale Client-Installation im internen Netzwerk über einen Webbrowser erreichbar sein.
2. Die Anwendung muss eine klare Trennung von Datenmodell, Fachlogik und Web-Oberfläche besitzen, damit sie wartbar erweitert werden kann.
3. Die Anwendung muss fehlerhafte Eingaben frühzeitig erkennen und dem Benutzer verständlich zurückmelden.
4. Die Anwendung muss sicherheitsrelevante Zugriffsvorgänge angemessen absichern, insbesondere durch Passwort-Hashing, Session-Verwaltung und Schutz gegen wiederholte Fehlanmeldungen.
5. Die Anwendung soll für den vorgesehenen internen Benutzerkreis mit vertretbarem Administrationsaufwand betrieben werden können.

Abgrenzung

Im Projekt wird nicht der Spamfilter selbst entwickelt. Es geht um die Verwaltung der dafür benötigten Stamm- und Freigabedaten. Auch der endgültige Produktivbetrieb, eine umfangreiche Übernahme alter Daten aus Fremdsystemen und zusätzliche Spezial-Schnittstellen zu Mail-Systemen gehören nicht zum Projektumfang.

A.5 Pflichtenheft

Das Pflichtenheft konkretisiert die technische Umsetzung der im Lastenheft beschriebenen Anforderungen für das *AntispamAdminCenter*.

Systemübersicht

Die Anwendung wird als serverseitige Webanwendung mit Python 3.12, FastAPI, SQLAlchemy und SQLite umgesetzt. Die Fachlogik ist in Service-Module ausgelagert, die Oberfläche wird über HTML-Templates bereitgestellt. Der Zugriff erfolgt per Browser im internen Netzwerk.

Umzusetzende Funktionen

1. Firmenverwaltung

Firmen werden in einer zentralen Datenbank gespeichert und über eigene Formulare erstellt, bearbeitet und angezeigt.

2. Benutzerverwaltung

Benutzer werden mit Vorname, Nachname, E-Mail-Adresse, Rolle und optionaler Firmenzuordnung verwaltet. Root- und Admin-Benutzer können Benutzer anlegen und ändern.

3. Rechtekonzept

Das Rollenmodell unterscheidet mindestens zwischen Root, Admin und User. Root darf firmenübergreifend arbeiten. Admin und User sind auf zugeordnete Firmen und die dort zulässigen Funktionen begrenzt.

4. Mail-Adressverwaltung

Mail-Adressen werden firmenbezogen gespeichert, in Formularen validiert und über die Oberfläche pflegbar gemacht.

5. IP-Adressverwaltung

IP-Adressen werden firmenbezogen gespeichert, auf syntaktische Korrektheit geprüft und über die Oberfläche pflegbar gemacht.

6. Authentifizierung

Die Anmeldung erfolgt über E-Mail-Adresse und Passwort. Passwörter werden mit Argon2 gehasht. Nach erfolgreicher Anmeldung wird eine Session erzeugt.

7. Login-Schutz

Fehlgeschlagene Anmeldungen werden protokolliert. Zusätzlich werden Rate-Limiting und kontobezogene Sperrmechanismen gegen wiederholte Fehlanmeldungen umgesetzt.

8. Audit-Log

Erstellen, Ändern und Löschen fachlicher Datensätze werden mit Benutzerbezug,

Aktion, Objektart und Änderungsdaten protokolliert. Sensible Felder wie Passwörter werden im Audit-Log maskiert.

9. **Rollback-Unterstützung**

Für ausgewählte Audit-Einträge werden die erforderlichen Informationen für ein kontrolliertes Wiederherstellen des vorherigen Zustands gespeichert.

10. **Datenbereitstellung**

Die Anwendung stellt die verwalteten Fachobjekte sowohl über browserbasierte Oberflächen als auch über schlanke API-Endpunkte für nachgelagerte Verarbeitung bereit.

Qualitätsanforderungen

1. Die Fachlogik muss vom Präsentationslayer getrennt implementiert werden.
2. Eingaben müssen serverseitig validiert werden.
3. Sicherheitsrelevante Vorgänge müssen nachvollziehbar protokolliert werden.
4. Die wichtigsten Funktionen müssen durch automatisierte Tests abgesichert werden.
5. Die Konfiguration soll über zentrale Konfigurationswerte erfolgen.

Betriebsbedingungen

1. Einsatz im internen Netzwerk ohne öffentlichen Internetzugriff
2. Bereitstellung auf einem System mit Python-Laufzeitumgebung
3. Persistente Datenspeicherung in einer SQLite-Datenbank
4. Nutzung durch interne Administratoren und autorisierte Firmenbenutzer

A.6 Use Case-Diagramm

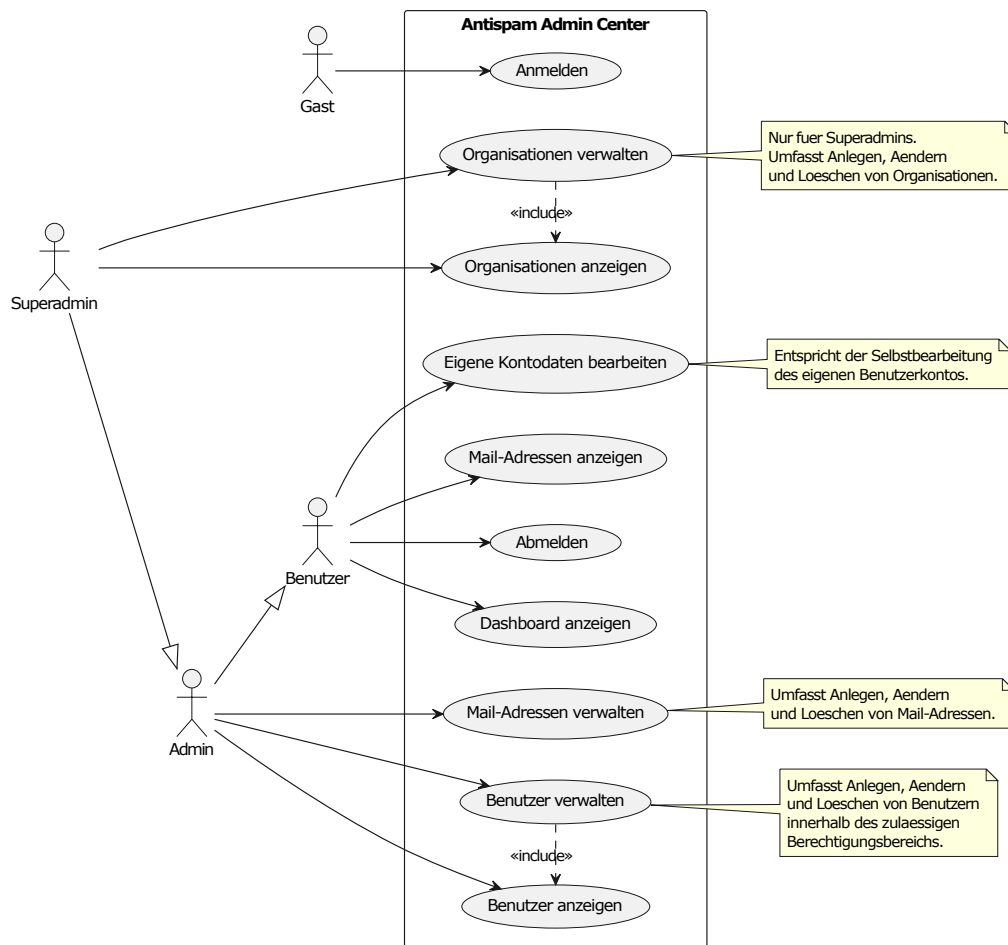


Abbildung 1: Use Case-Diagramm

A.7 Datenbankmodell

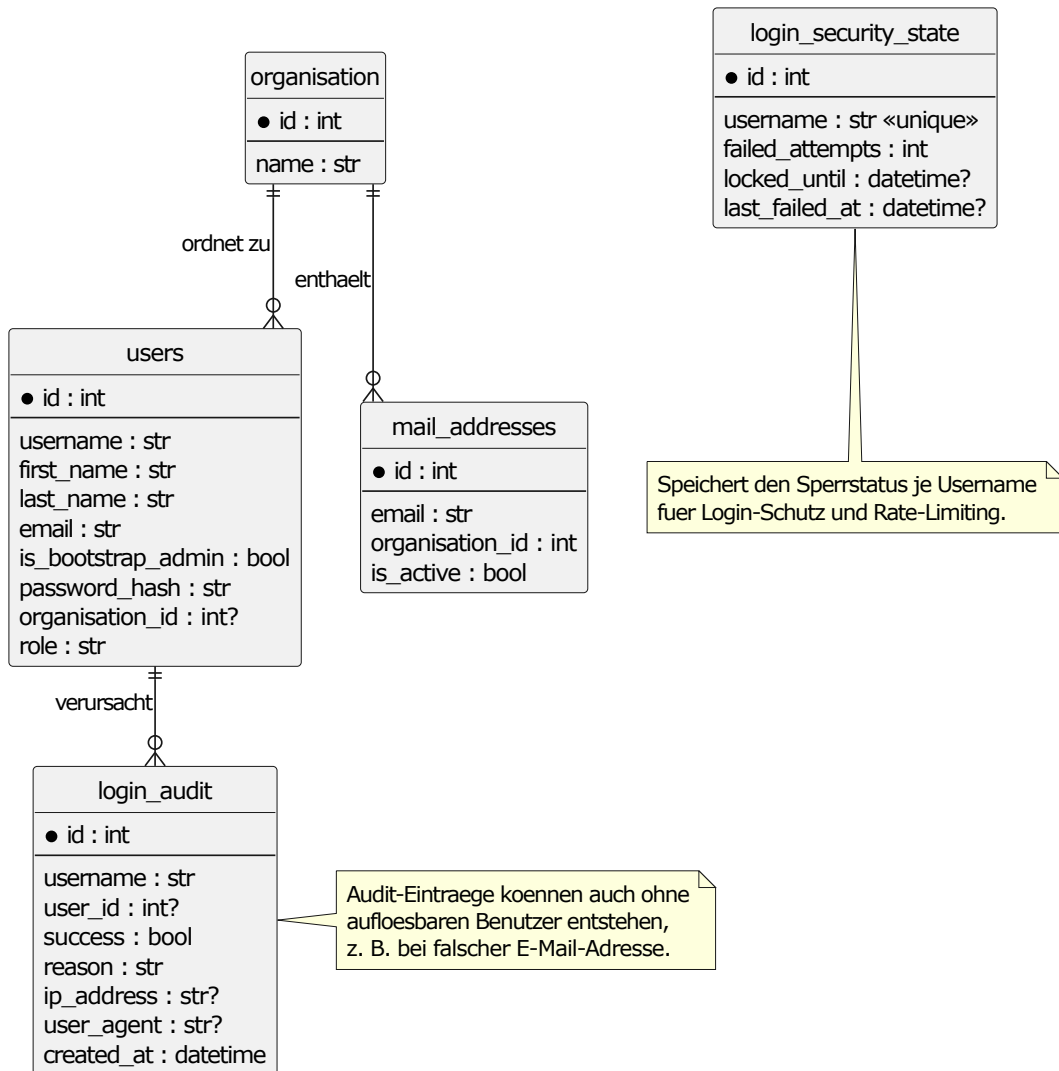


Abbildung 2: Datenbankmodell

A.8 Klassendiagramm

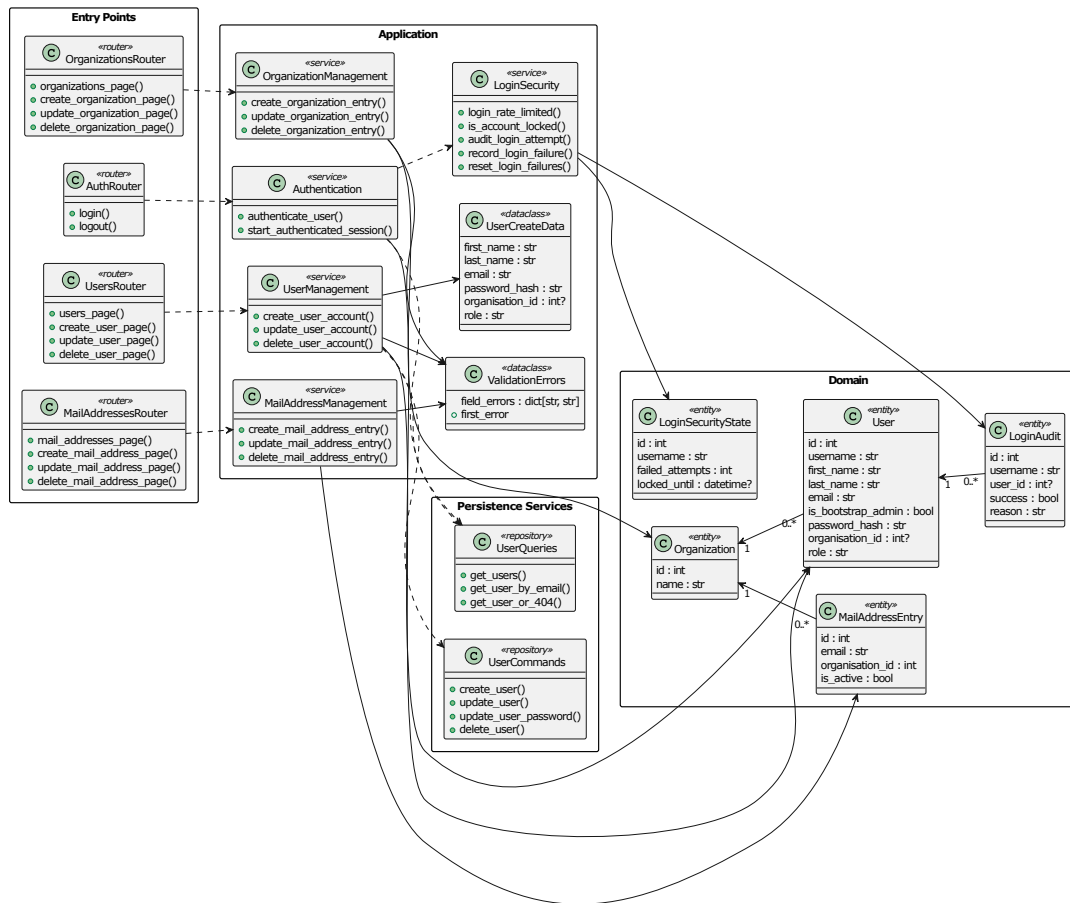


Abbildung 3: Klassendiagramm

A.9 Komponentendiagramm

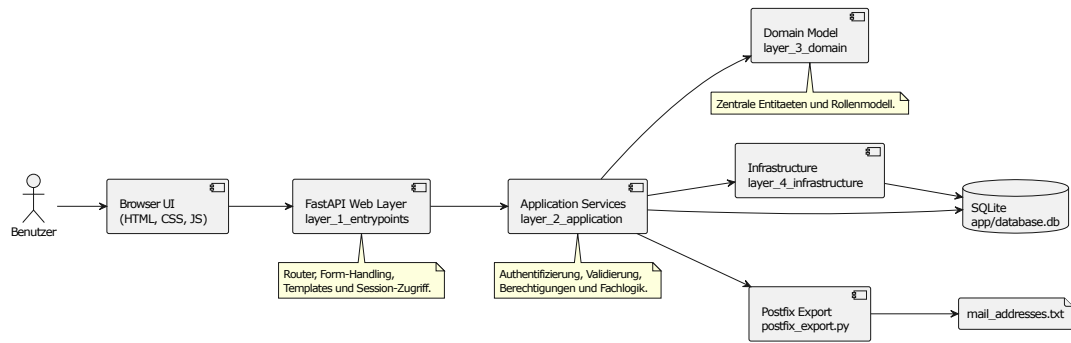


Abbildung 4: Komponentendiagramm

A.10 Sequenzdiagramm Login

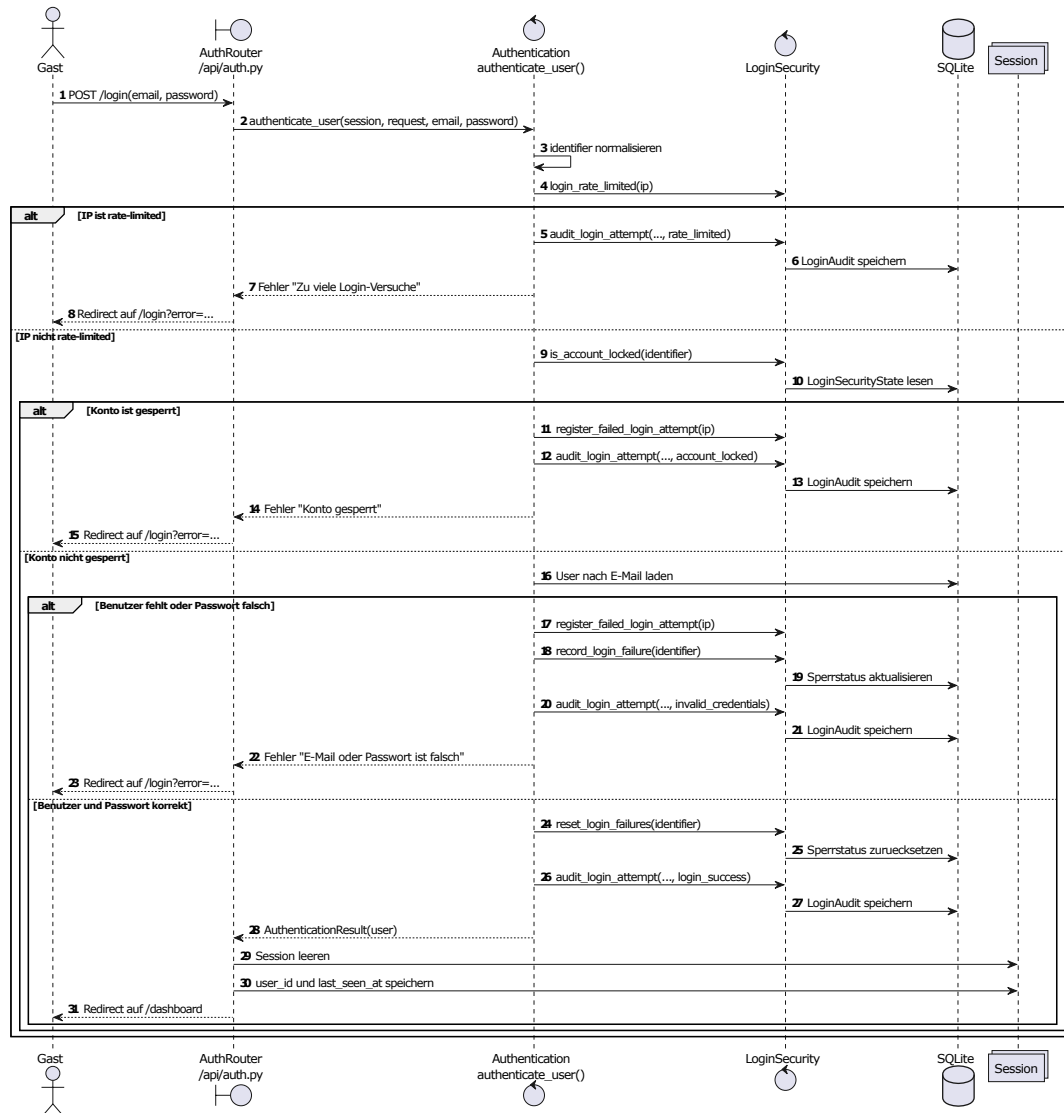


Abbildung 5: Sequenzdiagramm Login

A.11 Kundenhinweis

Die Anwendung ist für die tägliche Pflege von Firmen, Benutzern, Mail-Adressen und IP-Adressen vorgesehen. Vor dem Löschen eines Datensatzes soll geprüft werden, ob der Eintrag noch für nachgelagerte Prozesse benötigt wird. Fachliche Änderungen sind über das Audit-Log nachvollziehbar.

A.12 Quelltextauszug zu Datenschutz und Sicherheit

Die folgenden Auszüge zeigen die sicherheitsrelevanten Kernfunktionen der Anwendung. Sie sind nicht als vollständiger Quelltextabdruck gedacht, sondern folgen dem Ablauf eines geschützten Pflegevorgangs: Anmeldung, Prüfung der Berechtigung, Änderung von Daten, Protokollierung und optionales Zurücksetzen.

Anmeldung und Session

Der Einstieg in die Anwendung erfolgt über die Login-Funktion. Die E-Mail-Adresse wird normalisiert, danach werden Rate-Limit, Kontosperrung und Passwortprüfung durchlaufen. Erst bei erfolgreicher Prüfung wird eine neue Session gestartet.

```
25 def authenticate_user(  
26     *, session: Session, request: Request, email: str, password: str  
27 ) -> AuthenticationResult:  
28  
29     identifier = email.strip().lower()  
30     ip_address = request.client.host if request.client else None  
31  
32     if login_rate_limited(ip_address):  
33         audit_login_attempt(  
34             session=session,  
35             username=identifier,  
36             success=False,  
37             reason="rate_limited",  
38             request=request,  
39         )  
40         return AuthenticationResult(  
41             user=None,  
42             error_message="Zu_viele_Login-Versuche._Bitte_warten_Sie_kurz.",  
43         )  
44  
45     if is_account_locked(identifier, session):  
46         register_failed_login_attempt(ip_address)  
47         audit_login_attempt(  
48             session=session,  
49             username=identifier,  
50             success=False,  
51             reason="account_locked",  
52             request=request,  
53         )  
54         return AuthenticationResult(  
55             user=None,  
56             error_message="Konto_vorübergehend_gesperrt.",  
57         )  
58
```

```

59     statement = select(User).where(User.email == identifier)
60     user = session.exec(statement).first()
61
62     if user is None or not verify_password(password, user.password_hash):
63         register_failed_login_attempt(ip_address)
64         record_login_failure(identifier, session)
65         audit_login_attempt(
66             session=session,
67             username=identifier,
68             success=False,
69             reason="invalid_credentials",
70             request=request,
71             user_id=user.id if user is not None else None,
72         )
73         return AuthenticationResult(
74             user=None,
75             error_message="E-Mail_oder_Passwort_ist_falsch.",
76         )
77
78     reset_login_failures(identifier, session)
79     audit_login_attempt(
80         session=session,
81         username=identifier,
82         success=True,
83         reason="login_success",
84         request=request,
85         user_id=user.id,
86     )
87     return AuthenticationResult(user=user, error_message=None)
88
89
90 def start_authenticated_session(request: Request, user: User) -> None:
91     request.session.clear()

```

Listing 1: Authentifizierung und Start der Benutzersession

Passwortschutz

Passwörter werden nicht im Klartext gespeichert. Beim Anlegen oder Ändern eines Benutzers wird das Passwort gehasht. Zusätzlich gelten Mindestanforderungen an die Passwortstärke.

```
1 from argon2 import PasswordHasher
2 from argon2.exceptions import InvalidHashError, VerifyMismatchError,
  VerificationError
3
4 password_hasher = PasswordHasher()
5
6
7 def hash_password(password: str) -> str:
8     return password_hasher.hash(password)
9
10
11 def verify_password(password: str, stored_password_hash: str) -> bool:
12     try:
13         return password_hasher.verify(stored_password_hash, password)
14     except (InvalidHashError, VerifyMismatchError, VerificationError):
15         return stored_password_hash == password
16
17
18 def validate_password_strength(password: str) -> str | None:
19     if len(password) < 12:
20         return "Passwort_muss_mindestens_12_Zeichen_lang_sein."
21     if password.lower() == password or password.upper() == password:
22         return "Passwort_muss_Groß- und Kleinbuchstaben_enthalten."
23     if not any(character.isdigit() for character in password):
24         return "Passwort_muss_mindestens_eine_Zahl_enthalten."
25     if not any(not character.isalnum() for character in password):
26         return "Passwort_muss_mindestens_ein_Sonderzeichen_enthalten."
27     return None
```

Listing 2: Hashing und Prüfung der Passwortstärke

Schutz gegen wiederholte Fehlanmeldungen

Die Login-Sicherheit arbeitet auf zwei Ebenen. Wiederholte Fehlversuche von einer IP-Adresse werden zeitlich begrenzt gezählt. Zusätzlich wird je Konto gespeichert, ob eine Sperre aktiv ist. Jeder Login-Versuch wird protokolliert.

```
19 def register_failed_login_attempt(ip_address: str | None) -> None:
20     if ip_address is None:
21         return
22     attempts = login_attempts_by_ip[ip_address]
23     now = utc_now()
24     while attempts and now - attempts[0] > LOGIN_RATE_LIMIT_WINDOW:
25         attempts.popleft()
26     attempts.append(now)
27
28
29 def login_rate_limited(ip_address: str | None) -> bool:
30     if ip_address is None:
31         return False
32     attempts = login_attempts_by_ip[ip_address]
33     now = utc_now()
34     while attempts and now - attempts[0] > LOGIN_RATE_LIMIT_WINDOW:
35         attempts.popleft()
36     return len(attempts) >= LOGIN_RATE_LIMIT_COUNT
37
38
39 def get_login_state(username: str, session: Session) -> LoginSecurityState
40     :
41     statement = select(LoginSecurityState).where(LoginSecurityState.
42         username == username)
43     state = session.exec(statement).first()
44     if state is None:
45         state = LoginSecurityState(username=username)
46     return state
47
48
49 def is_account_locked(username: str, session: Session) -> bool:
50     state = get_login_state(username, session)
51     return state.locked_until is not None and state.locked_until > utc_now
52         ()
53
54
55 def record_login_failure(username: str, session: Session) -> None:
56     state = get_login_state(username, session)
57     state.failed_attempts += 1
58     state.last_failed_at = utc_now().replace(tzinfo=None)
59     if state.failed_attempts >= LOGIN_LOCKOUT_THRESHOLD:
```

```

57         state.locked_until = (utc_now() + LOGIN_LOCKOUT_DURATION).replace(
58             tzinfo=None)
59     session.add(state)
60     session.commit()
61
62 def reset_login_failures(username: str, session: Session) -> None:
63     state = get_login_state(username, session)
64     state.failed_attempts = 0
65     state.locked_until = None
66     state.last_failed_at = None
67     session.add(state)
68     session.commit()
69
70
71 def audit_login_attempt(
72     *,
73     session: Session,
74     username: str,
75     success: bool,
76     reason: str,
77     request: Request,
78     user_id: int | None = None,
79 ) -> None:
80     audit_entry = LoginAudit(
81         username=username,
82         user_id=user_id,
83         success=success,
84         reason=reason,
85         ip_address=request.client.host if request.client else None,
86         user_agent=request.headers.get("user-agent"),
87     )
88     session.add(audit_entry)
89     session.commit()

```

Listing 3: Rate-Limit, Kontosperrung und Login-Audit

Rollen und Mandantentrennung

Die Mandantentrennung wird nicht nur in der Oberfläche abgebildet. Die Service-Funktionen filtern Datensätze anhand der Rolle und der zugeordneten Firma. Benutzer ohne Root-Rolle sehen dadurch nur Daten aus ihrer Firma.

```
8 def ensure_user_has_roles(current_user: User, *roles: str) -> User:
9     if current_user.role not in roles:
10         raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="
            Forbidden")
11     return current_user
12
13
14 def has_users(session: Session) -> bool:
15     return session.exec(select(User)).first() is not None
16
17
18 def is_root(user: User) -> bool:
19     return user.role == ROLE_ROOT
20
21
22 def is_admin(user: User) -> bool:
23     return user.role == ROLE_ADMIN
24
25
26 def is_manager(user: User) -> bool:
27     return user.role in {ROLE_ROOT, ROLE_ADMIN}
28
29
30 def get_accessible_organizations(
31     session: Session, current_user: User
32 ) -> list[Organization]:
33     if is_root(current_user):
34         return list(session.exec(select(Organization)).all())
35     if current_user.organization_id is None:
36         return []
37     organization = session.get(Organization, current_user.organization_id)
38     return [organization] if organization is not None else []
39
40
41 def get_accessible_mail_addresses(
42     session: Session, current_user: User
43 ) -> list[MailAddressEntry]:
44     statement = select(MailAddressEntry)
45     if not is_root(current_user):
46         statement = statement.where(
47             MailAddressEntry.organization_id == current_user.organization_id
48         )
```

```

49     return list(session.exec(statement).all())
50
51
52 def get_accessible_ip_addresses(
53     session: Session, current_user: User
54 ) -> list[IPAddressEntry]:
55     statement = select(IPAddressEntry)
56     if not is_root(current_user):
57         statement = statement.where(
58             IPAddressEntry.organization_id == current_user.organization_id
59         )
60     return list(session.exec(statement).all())
61
62
63 def get_accessible_users(session: Session, current_user: User) -> list[
64     User]:
65     if current_user.role == ROLE_USER:
66         return [current_user]
67     statement = select(User)
68     if not is_root(current_user):
69         statement = statement.where(User.organization_id == current_user.
70             organization_id)
71     return list(session.exec(statement).all())

```

Listing 4: Rollenprüfung und Filterung sichtbarer Daten

Schutz vor firmenfremden Änderungen

Vor jeder Änderung wird geprüft, ob der angemeldete Benutzer den betroffenen Datensatz bearbeiten darf. Root darf firmenübergreifend arbeiten, andere Benutzer bleiben auf ihre Firma begrenzt. Root-Benutzer werden zusätzlich vor versehentlicher Bearbeitung durch andere Rollen geschützt.

```
78 def ensure_manageable_organization(  
79     current_user: User, organization_id: int | None  
80 ) -> None:  
81     if is_root(current_user):  
82         return  
83     if current_user.organization_id is None or current_user.organization_id  
84         != organization_id:  
85         raise HTTPException(  
86             status_code=status.HTTP_403_FORBIDDEN,  
87             detail="Forbidden",  
88         )  
89  
90 def ensure_manageable_mail_address(current_user: User, entry:  
91     MailAddressEntry) -> None:  
92     ensure_manageable_organization(current_user, entry.organization_id)  
93  
94 def ensure_manageable_ip_address(current_user: User, entry: IPAddressEntry  
95     ) -> None:  
96     ensure_manageable_organization(current_user, entry.organization_id)  
97  
98 def ensure_manageable_user(current_user: User, target_user: User) -> None:  
99     if target_user.is_root:  
100         raise HTTPException(  
101             status_code=status.HTTP_403_FORBIDDEN,  
102             detail="Forbidden",  
103         )  
104     if is_root(current_user):  
105         return  
106     if target_user.role == ROLE_ROOT:  
107         raise HTTPException(  
108             status_code=status.HTTP_403_FORBIDDEN,  
109             detail="Forbidden",  
110         )  
111     ensure_manageable_organization(current_user, target_user.  
        organization_id)
```

Listing 5: Prüfung bearbeitbarer Firmen und Benutzer

Auditierung und Schutz sensibler Daten

Fachliche Änderungen werden im Audit-Log gespeichert. Dabei werden sensible Felder wie Passwörter vor dem Speichern maskiert. Dadurch bleibt die Änderung nachvollziehbar, ohne Passwortwerte im Audit-Log offenzulegen.

```
9 SENSITIVE_AUDIT_FIELDS = {"password", "password_hash"}
10 REDACTED_AUDIT_VALUE = "****"
11
12 AUDIT_ENTITY_LABELS = {
13     "organization": "Organisation",
14     "user": "Benutzer",
15     "mail_address": "Mail-Adresse",
16     "ip_address": "IP-Adresse",
17 }
18
19 ENTITY_MODEL_MAP = {
20     "organization": Organization,
21     "user": User,
22     "mail_address": MailAddressEntry,
23     "ip_address": IPAddressEntry,
24 }
25
26
27 def _json_default(value: object) -> str:
28     if isinstance(value, datetime):
29         return value.isoformat()
30     return str(value)
31
32
33 def redact_audit_payload(value: object) -> object:
34     if isinstance(value, dict):
35         return {
36             key: (
37                 REDACTED_AUDIT_VALUE
38                 if key in SENSITIVE_AUDIT_FIELDS
39                 else redact_audit_payload(nested_value)
40             )
41             for key, nested_value in value.items()
42         }
43     if isinstance(value, list):
44         return [redact_audit_payload(item) for item in value]
45     return value
46
47
48 def dumps_payload(payload: dict[str, object]) -> str:
49     return json.dumps(payload, ensure_ascii=False, default=_json_default,
50                        sort_keys=True)
```

Listing 6: Maskierung sensibler Felder im Audit-Log

```
121 def create_audit_log(  
122     *,  
123     session: Session,  
124     actor: User,  
125     entity_type: str,  
126     entity_id: int | None,  
127     action: str,  
128     organization_id: int | None,  
129     changes: dict[str, object],  
130     rollback: dict[str, object] | None,  
131 ) -> AuditLog:  
132     audit_entry = AuditLog(  
133         entity_type=entity_type,  
134         entity_id=entity_id,  
135         action=action,  
136         actor_user_id=get_actor_id(actor),  
137         actor_email=get_actor_email(actor),  
138         organization_id=organization_id,  
139         changes_json=dumps_payload(redact_audit_payload(changes)),  
140         rollback_json=dumps_payload(rollback) if rollback is not None else  
141             None,  
142     )  
143     session.add(audit_entry)  
144     session.commit()  
145     session.refresh(audit_entry)  
146     return audit_entry
```

Listing 7: Erzeugen eines Audit-Eintrags

Audit-Einträge für Änderungen

Für Erstellen, Ändern und Löschen werden jeweils passende Audit-Informationen abgelegt. Bei Änderungen wird sowohl der vorherige als auch der neue Zustand gespeichert. Dadurch kann später nachvollzogen werden, was konkret geändert wurde.

```
148 def record_create_audit(*, session: Session, actor: User, entity: SQLAlchemyModel
    ) -> AuditLog:
149     snapshot = snapshot_entity(entity)
150     return create_audit_log(
151         session=session,
152         actor=actor,
153         entity_type=get_entity_type(entity),
154         entity_id=snapshot.get("id"),
155         action="create",
156         organization_id=snapshot.get("organization_id"),
157         changes={"after": snapshot},
158         rollback={
159             "operation": "delete",
160             "entity_type": get_entity_type(entity),
161             "entity_id": snapshot.get("id"),
162         },
163     )
164
165
166 def record_update_audit(
167     *,
168     session: Session,
169     actor: User,
170     entity: SQLAlchemyModel,
171     before: dict[str, object],
172 ) -> AuditLog:
173     after = snapshot_entity(entity)
174     return create_audit_log(
175         session=session,
176         actor=actor,
177         entity_type=get_entity_type(entity),
178         entity_id=after.get("id"),
179         action="update",
180         organization_id=after.get("organization_id") or before.get("
            organization_id"),
181         changes={"before": before, "after": after},
182         rollback={
183             "operation": "restore_fields",
184             "entity_type": get_entity_type(entity),
185             "entity_id": after.get("id"),
186             "before": before,
187         },
```



```

188     )
189
190
191 def record_delete_audit(
192     *,
193     session: Session,
194     actor: User,
195     entity_type: str,
196     entity_id: int | None,
197     before: dict[str, object],
198     organization_id: int | None,
199 ) -> AuditLog:
200     return create_audit_log(
201         session=session,
202         actor=actor,
203         entity_type=entity_type,
204         entity_id=entity_id,
205         action="delete",
206         organization_id=organization_id,
207         changes={"before": before},
208         rollback={
209             "operation": "recreate",
210             "entity_type": entity_type,
211             "before": before,
212         },
213     )

```

Listing 8: Audit-Einträge für Erstellen, Ändern und Löschen

Kontrolliertes Zurücksetzen

Für ausgewählte Audit-Einträge kann der vorherige Zustand wiederhergestellt werden. Auch dieser Vorgang erzeugt wieder einen neuen Audit-Eintrag, damit das Zurücksetzen selbst nachvollziehbar bleibt.

```
286 def rollback_audit_entry(*, session: Session, audit_entry: AuditLog, actor
    : User) -> None:
287     rollback = loads_payload(audit_entry.rollback_json)
288     if rollback is None:
289         raise ValueError("No rollback data available.")
290
291     operation = rollback.get("operation")
292     entity_type = rollback.get("entity_type")
293     model = ENTITY_MODEL_MAP.get(entity_type)
294     if model is None:
295         raise ValueError("Unsupported rollback entity type.")
296
297     if operation == "delete":
298         entity_id = rollback.get("entity_id")
299         entity = session.get(model, entity_id)
300         if entity is None:
301             raise ValueError("Entity for rollback no longer exists.")
302         before = snapshot_entity(entity)
303         session.delete(entity)
304         session.commit()
305         create_audit_log(
306             session=session,
307             actor=actor,
308             entity_type=entity_type,
309             entity_id=entity_id,
310             action="rollback",
311             organization_id=before.get("organization_id"),
312             changes={"rolled_back_audit_id": audit_entry.id, "deleted":
                before},
313             rollback=None,
314         )
315         return
316
317     if operation == "restore_fields":
318         entity_id = rollback.get("entity_id")
319         before = rollback.get("before")
320         entity = session.get(model, entity_id)
321         if entity is None or not isinstance(before, dict):
322             raise ValueError("Entity for rollback no longer exists.")
323         previous_state = snapshot_entity(entity)
324         for key, value in before.items():
325             setattr(entity, key, value)
```

```

326     session.add(entity)
327     session.commit()
328     session.refresh(entity)
329     create_audit_log(
330         session=session,
331         actor=actor,
332         entity_type=entity_type,
333         entity_id=entity_id,
334         action="rollback",
335         organization_id=before.get("organization_id"),
336         changes={
337             "rolled_back_audit_id": audit_entry.id,
338             "before": previous_state,
339             "after": before,
340         },
341         rollback=None,
342     )
343     return
344
345 if operation == "recreate":
346     before = rollback.get("before")
347     if not isinstance(before, dict):
348         raise ValueError("No restore snapshot available.")
349     entity = model(**before)
350     session.add(entity)
351     session.commit()
352     session.refresh(entity)
353     create_audit_log(
354         session=session,
355         actor=actor,
356         entity_type=entity_type,
357         entity_id=entity.id,
358         action="rollback",
359         organization_id=before.get("organization_id"),
360         changes={"rolled_back_audit_id": audit_entry.id, "recreated":
361             before},
362         rollback=None,
363     )
364     return
365
366 raise ValueError("Unsupported rollback operation.")

```

Listing 9: Rollback auf Basis eines Audit-Eintrags

API-Beispiel

Die API nutzt dieselben Berechtigungsprüfungen wie die Weboberfläche. Im Beispiel dürfen nur Manager eine Mail-Adresse anlegen. Beim Lesen werden die sichtbaren Mail-Adressen über den aktuell angemeldeten Benutzer gefiltert.

```
21 @router.post("/", response_model=MailAddressPublic)
22 def create_mail_address_api(
23     entry: MailAddressCreate,
24     session: SessionDep,
25     current_user: ManagerUserDep,
26 ) -> MailAddressPublic:
27     result = create_mail_address_entry(
28         session,
29         current_user,
30         email=entry.email,
31         organization_id=entry.organization_id,
32         is_active=entry.is_active,
33     )
34     if result.error_message is not None or result.entry is None:
35         raise HTTPException(
36             status_code=status.HTTP_400_BAD_REQUEST,
37             detail={
38                 "message": result.error_message or "Ungültige Mail-Adresse.",
39                 "field_errors": (
40                     result.validation_errors.field_errors
41                     if result.validation_errors is not None
42                     else {}
43                 ),
44             },
45         )
46     return result.entry
47
48
49 @router.get("/", response_model=list[MailAddressPublic])
50 def read_mail_addresses(
51     session: SessionDep,
52     current_user: CurrentUserDep,
53 ) -> list[MailAddressPublic]:
54     return list_visible_mail_addresses(session, current_user)
```

Listing 10: API-Auszug für Mail-Adressen mit Rollenprüfung